Adversarial Development of Component Based Systems using AI

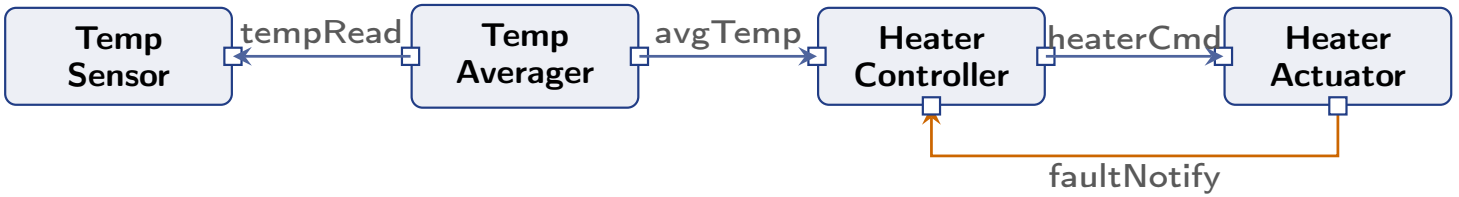
Robert Bocchino, **Klaus Havelund**, Nicolas Rouquette



Jet Propulsion Laboratory
California Institute of Technology

AISoLA 2026 — AI Assisted Programming

An LLM wrote this heater controller

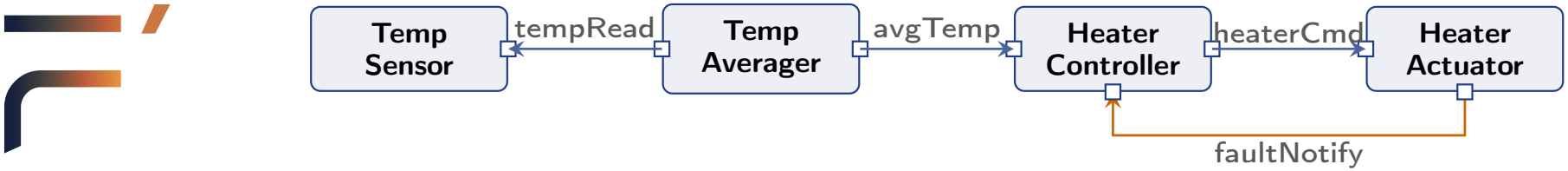


TempControl: four F′ components, connected through typed **ports**, keep a heater at a target temperature. The actuator stops the heater if it runs too long. It compiles, it runs, and all **38 unit tests pass**.

Lines of code (written for TempControl, not the F′ framework)	
C++, the 4 components	824
HeaterActuator 296, HeaterController 265, TempAverager 178, TempSensorReader 85	
C++, deployment setup	335
FPP: components, types, ports, topology	729
Unit tests	1,677

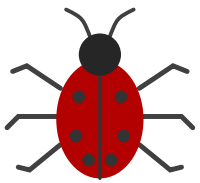
Yet it has bugs that testing did not catch

An LLM wrote this heater controller



TempControl: four F′ components, connected through typed **ports**, keep a heater at a target temperature. The actuator stops the heater if it runs too long. It compiles, it runs, and all **38 unit tests pass**.

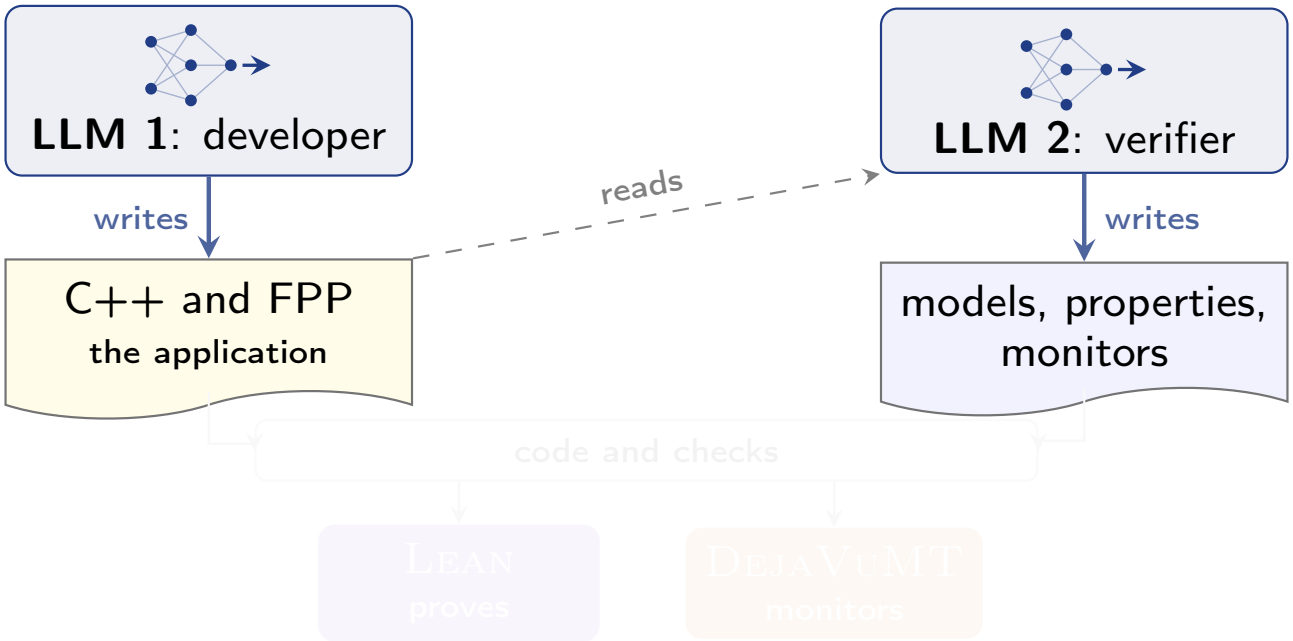
Lines of code (written for TempControl, not the F′ framework)	
C++, the 4 components	824
HeaterActuator 296, HeaterController 265, TempAverager 178, TempSensorReader 85	
C++, deployment setup	335
FPP: components, types, ports, topology	729
Unit tests	1,677



Yet it has bugs that testing did not catch

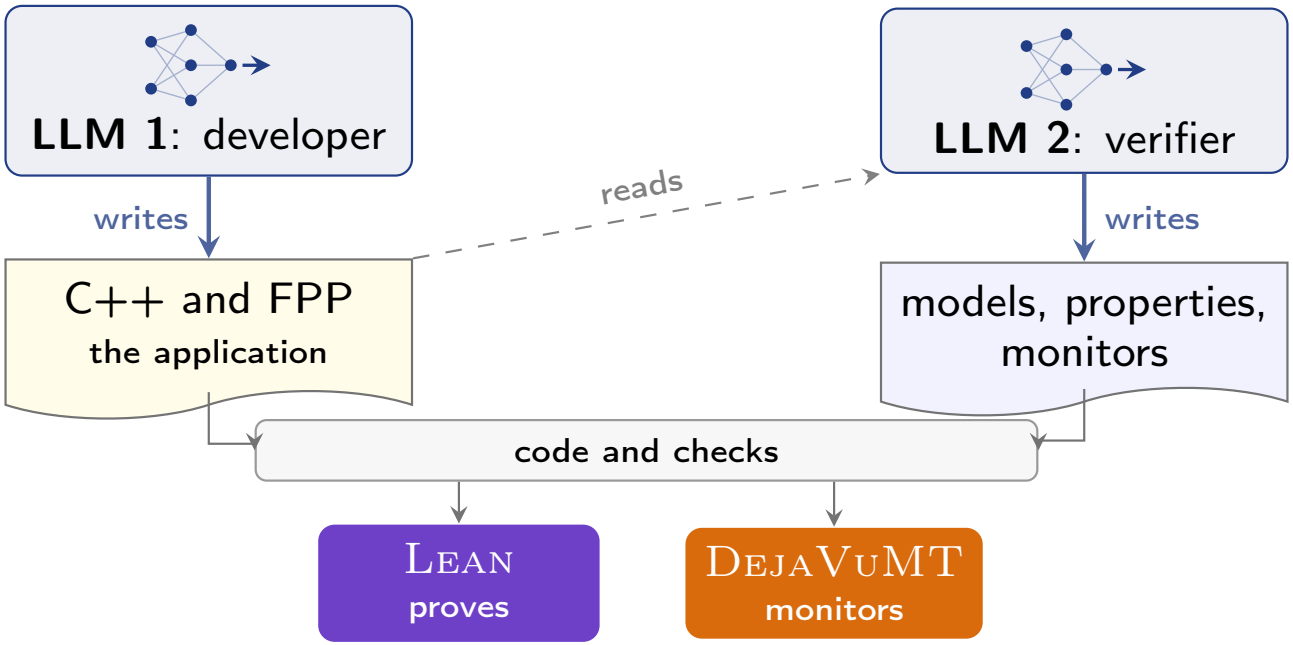
Adversarial development

One LLM writes the code, a second LLM writes the **checks** for it, and sound tools show where the two are inconsistent.



Adversarial development

One LLM writes the code, a second LLM writes the **checks** for it, and sound tools show where the two are inconsistent.

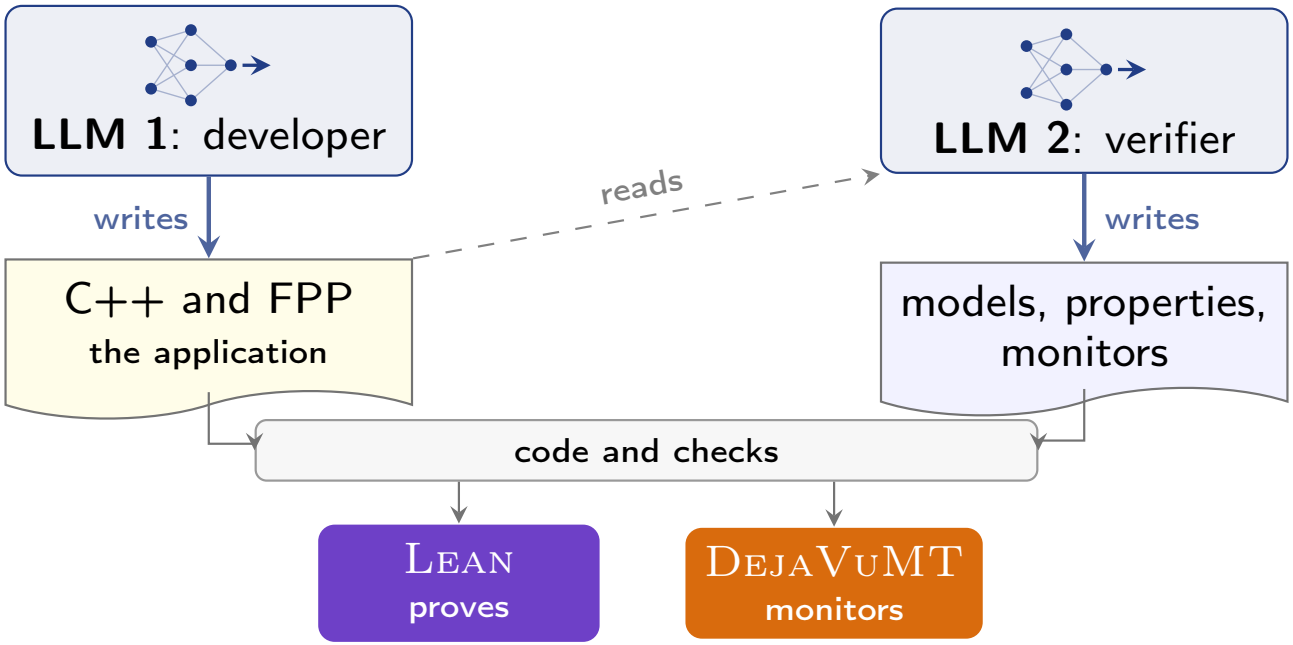


sound tools: neither LLM is trusted

The **disagreements** are the findings.

Adversarial development

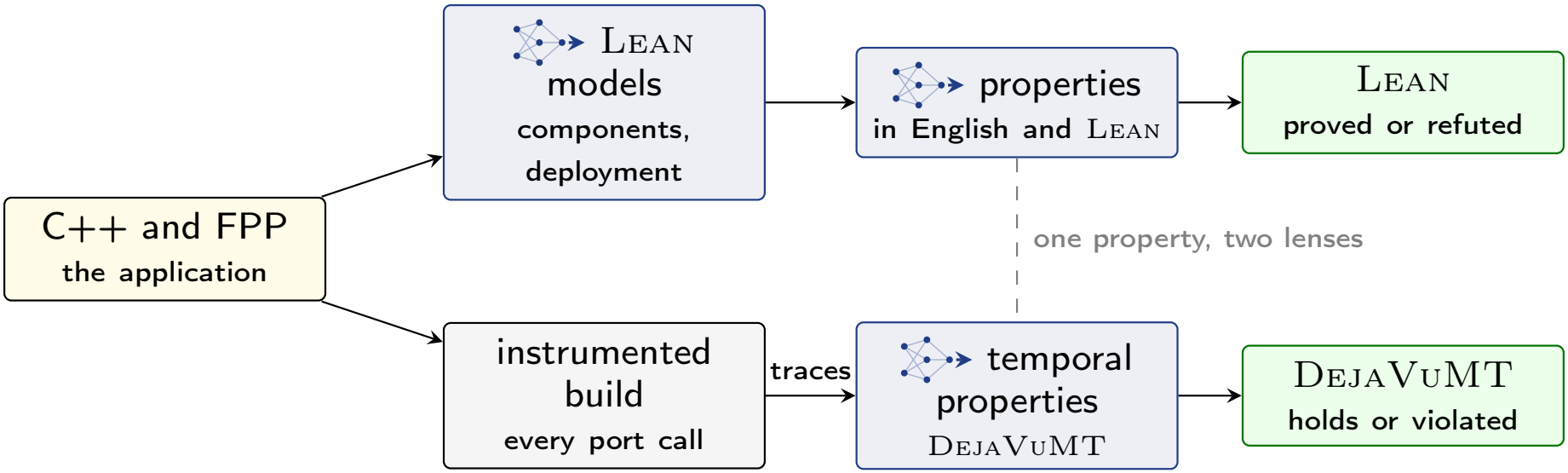
One LLM writes the code, a second LLM writes the **checks** for it, and sound tools show where the two are inconsistent.



sound tools: neither LLM is trusted

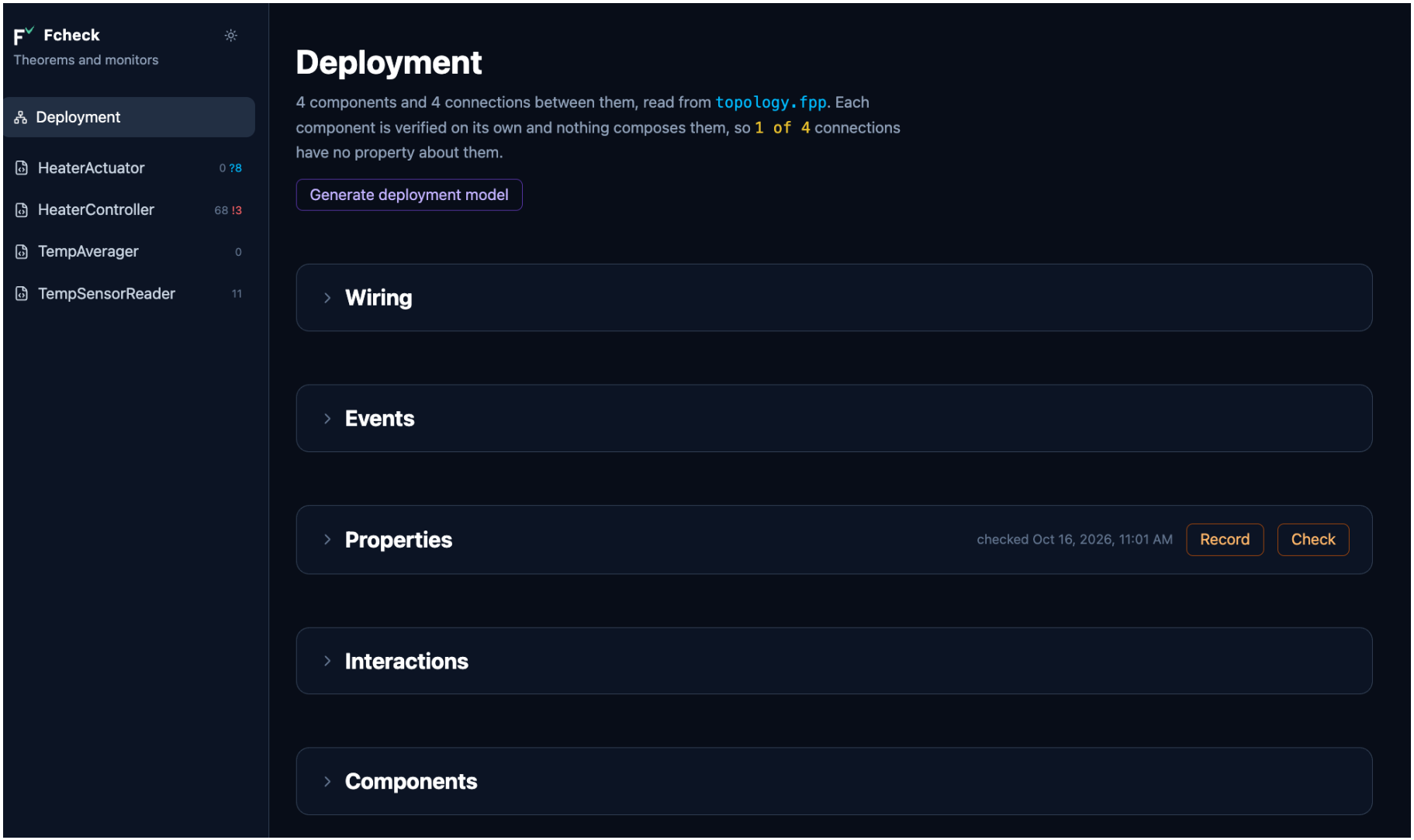
The **disagreements** are the findings.

F✓: the proof and monitor pipelines



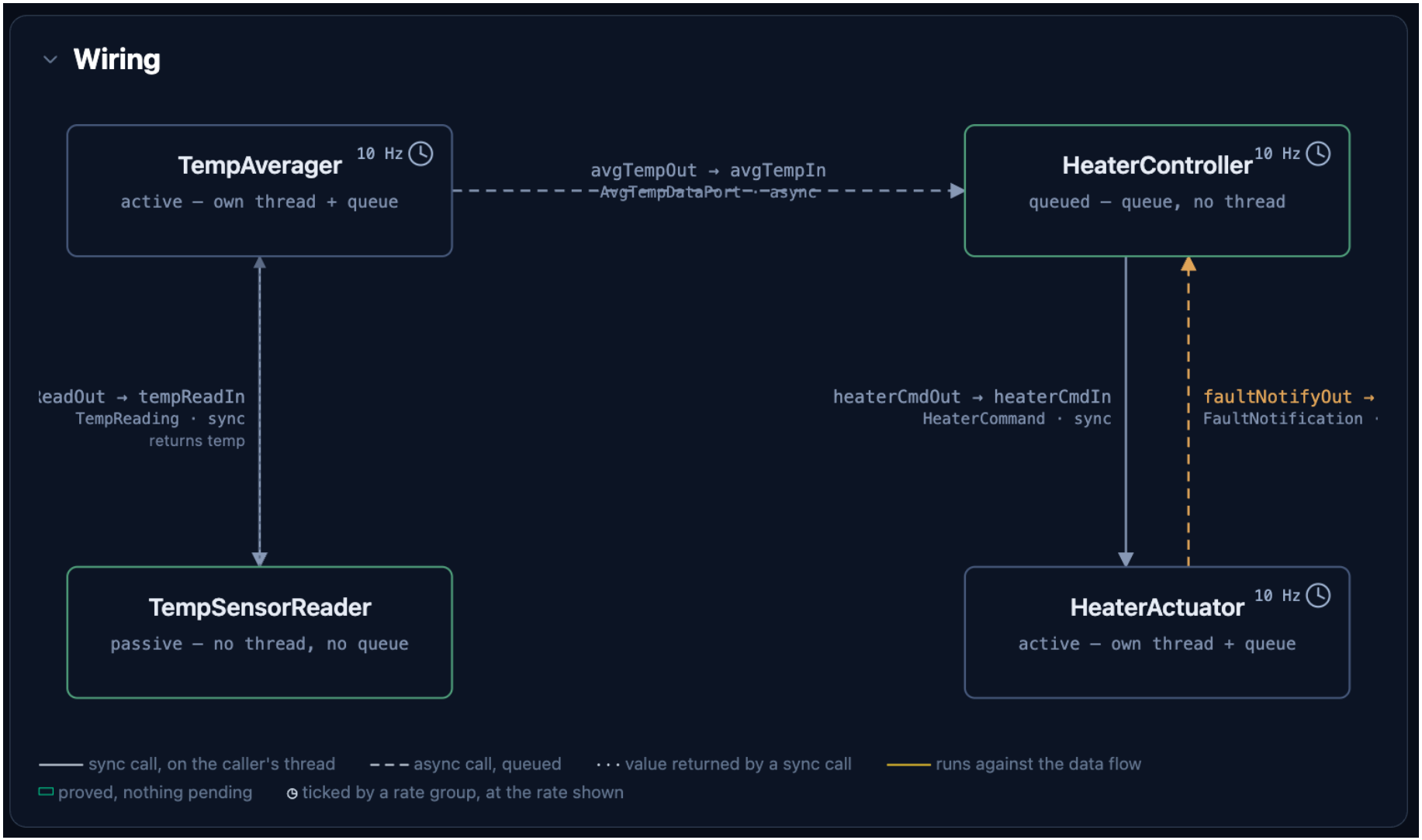
- Lean: a theorem prover; proofs are checked by its small kernel.
- DejaVuMT (work with Doron Peled): first-order past and future time logic with time bounds (`P[<=10]`, `F[<=1000]`), checked over a trace with an SMT solver.
- Works on **any** F′ application: reads the FPP, topology and dictionary.

The Deployment page



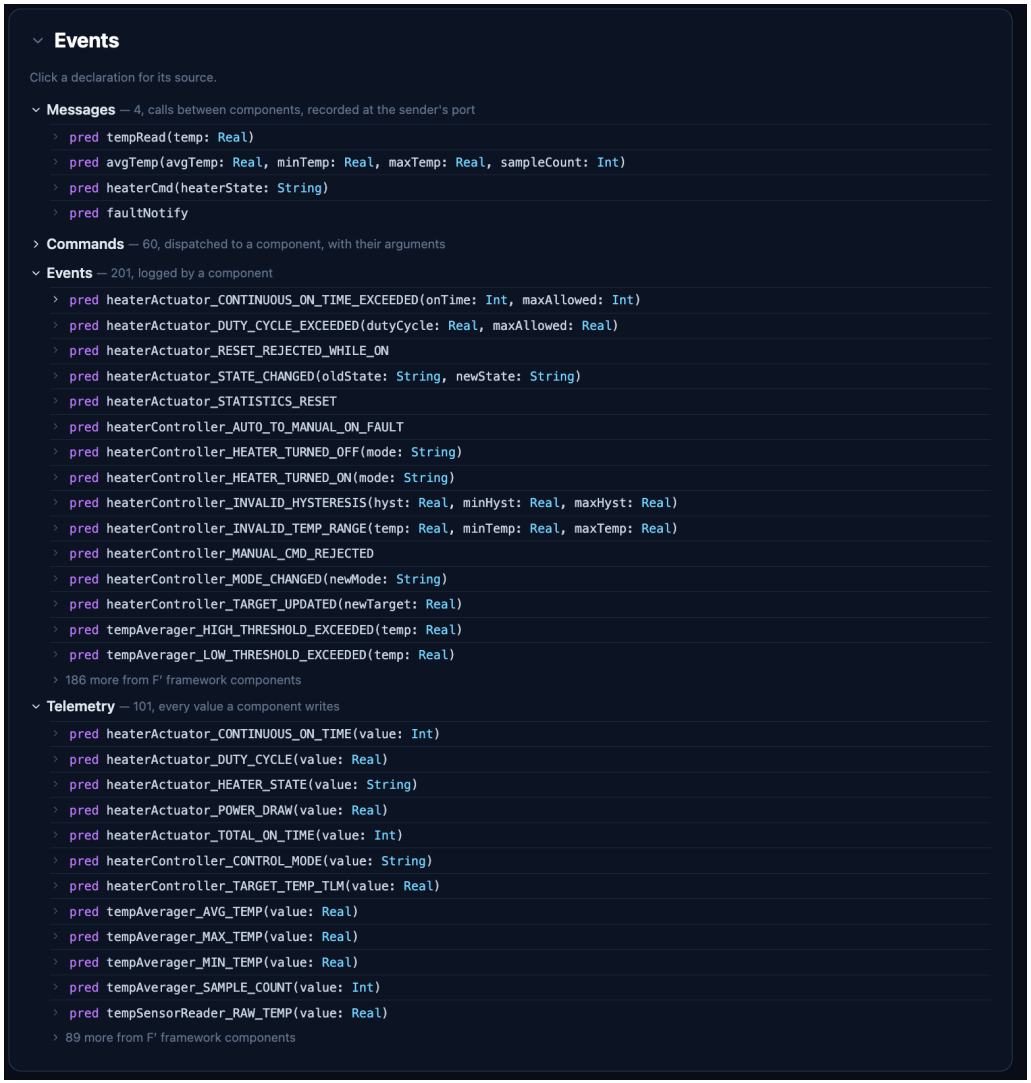
Generated from `topology.fpp`

The wiring



Read from the topology: sync and async ports, rate groups, data flow.

What temporal properties can talk about



Events

Click a declaration for its source.

Messages — 4, calls between components, recorded at the sender's port

- pred tempRead(temp: Real)
- pred avgTemp(avgTemp: Real, minTemp: Real, maxTemp: Real, sampleCount: Int)
- pred heaterCmd(heaterState: String)
- pred faultNotify

Commands — 60, dispatched to a component, with their arguments

Events — 201, logged by a component

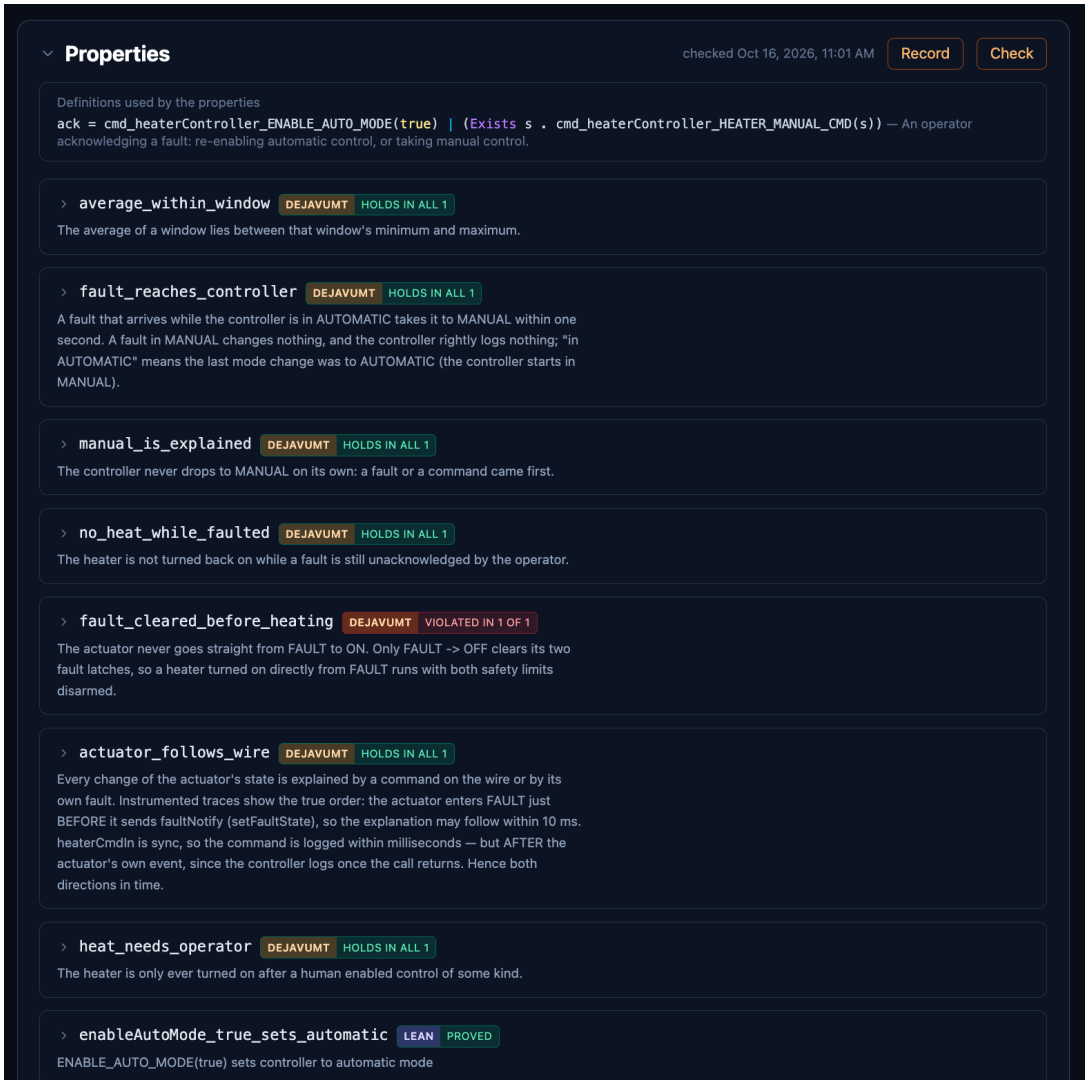
- pred heaterActuator_CONTINUOUS_ON_TIME_EXCEEDED(onTime: Int, maxAllowed: Int)
- pred heaterActuator_DUTY_CYCLE_EXCEEDED(dutyCycle: Real, maxAllowed: Real)
- pred heaterActuator_RESET_REJECTED_WHILE_ON
- pred heaterActuator_STATE_CHANGED(oldState: String, newState: String)
- pred heaterActuator_STATISTICS_RESET
- pred heaterController_AUTO_TO_MANUAL_ON_FAULT
- pred heaterController_HEATER_TURNED_OFF(mode: String)
- pred heaterController_HEATER_TURNED_ON(mode: String)
- pred heaterController_INVALID_HYSTERESIS(hyst: Real, minHyst: Real, maxHyst: Real)
- pred heaterController_INVALID_TEMP_RANGE(temp: Real, minTemp: Real, maxTemp: Real)
- pred heaterController_MANUAL_CMD_REJECTED
- pred heaterController_MODE_CHANGED(newMode: String)
- pred heaterController_TARGET_UPDATED(newTarget: Real)
- pred tempAverager_HIGH_THRESHOLD_EXCEEDED(temp: Real)
- pred tempAverager_LOW_THRESHOLD_EXCEEDED(temp: Real)
- 186 more from F′ framework components

Telemetry — 101, every value a component writes

- pred heaterActuator_CONTINUOUS_ON_TIME(value: Int)
- pred heaterActuator_DUTY_CYCLE(value: Real)
- pred heaterActuator_HEATER_STATE(value: String)
- pred heaterActuator_POWER_DRAW(value: Real)
- pred heaterActuator_TOTAL_ON_TIME(value: Int)
- pred heaterController_CONTROL_MODE(value: String)
- pred heaterController_TARGET_TEMP_TLM(value: Real)
- pred tempAverager_AVG_TEMP(value: Real)
- pred tempAverager_MAX_TEMP(value: Real)
- pred tempAverager_MIN_TEMP(value: Real)
- pred tempAverager_SAMPLE_COUNT(value: Int)
- pred tempSensorReader_RAW_TEMP(value: Real)
- 89 more from F′ framework components

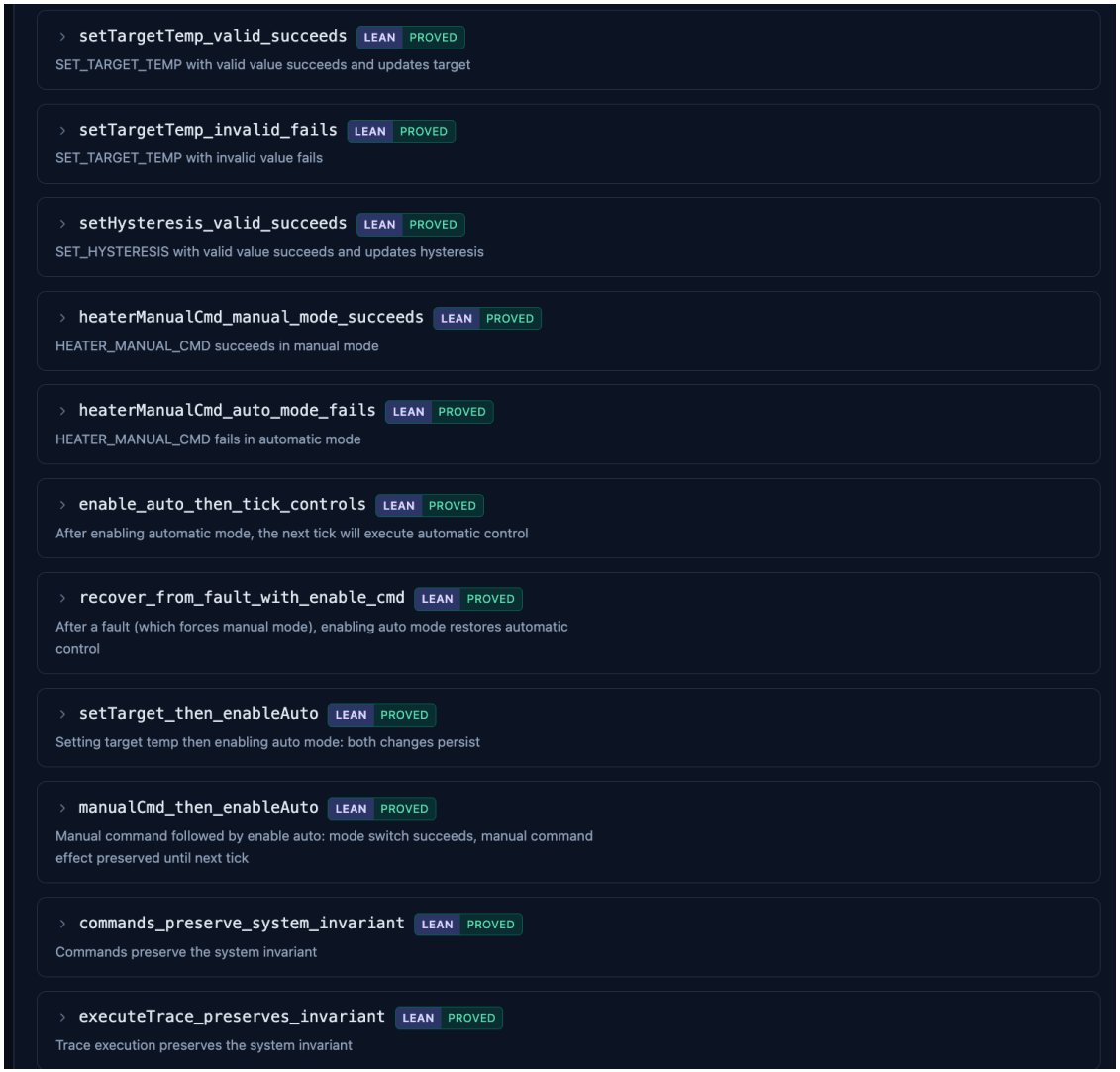
Generated from the F′ dictionary: every message, command, event and telemetry channel, typed.

Deployment properties: DejaVuMT



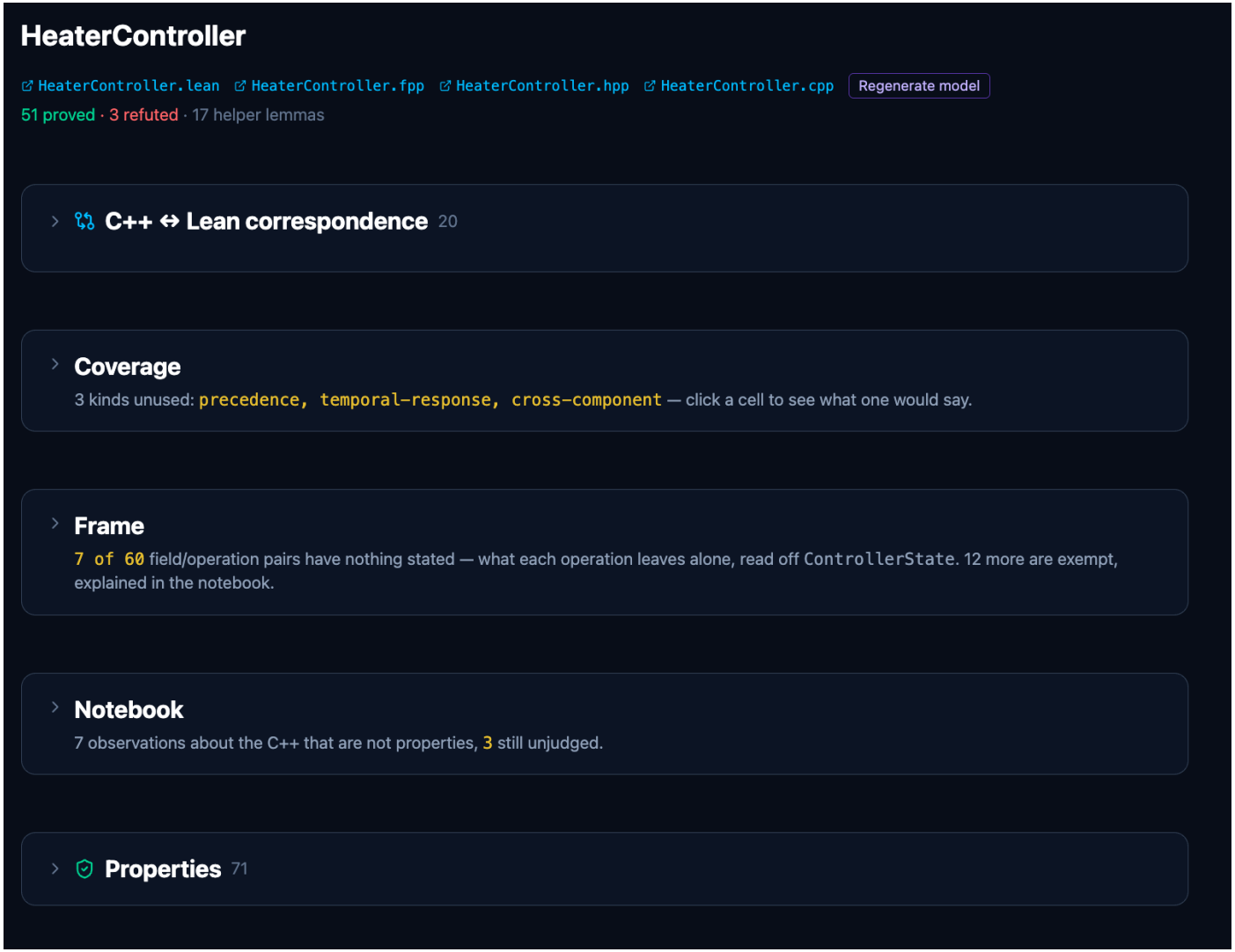
Checked on recorded runs of the instrumented system. One is violated.

Deployment properties: Lean



Theorems over the system model.

A component page



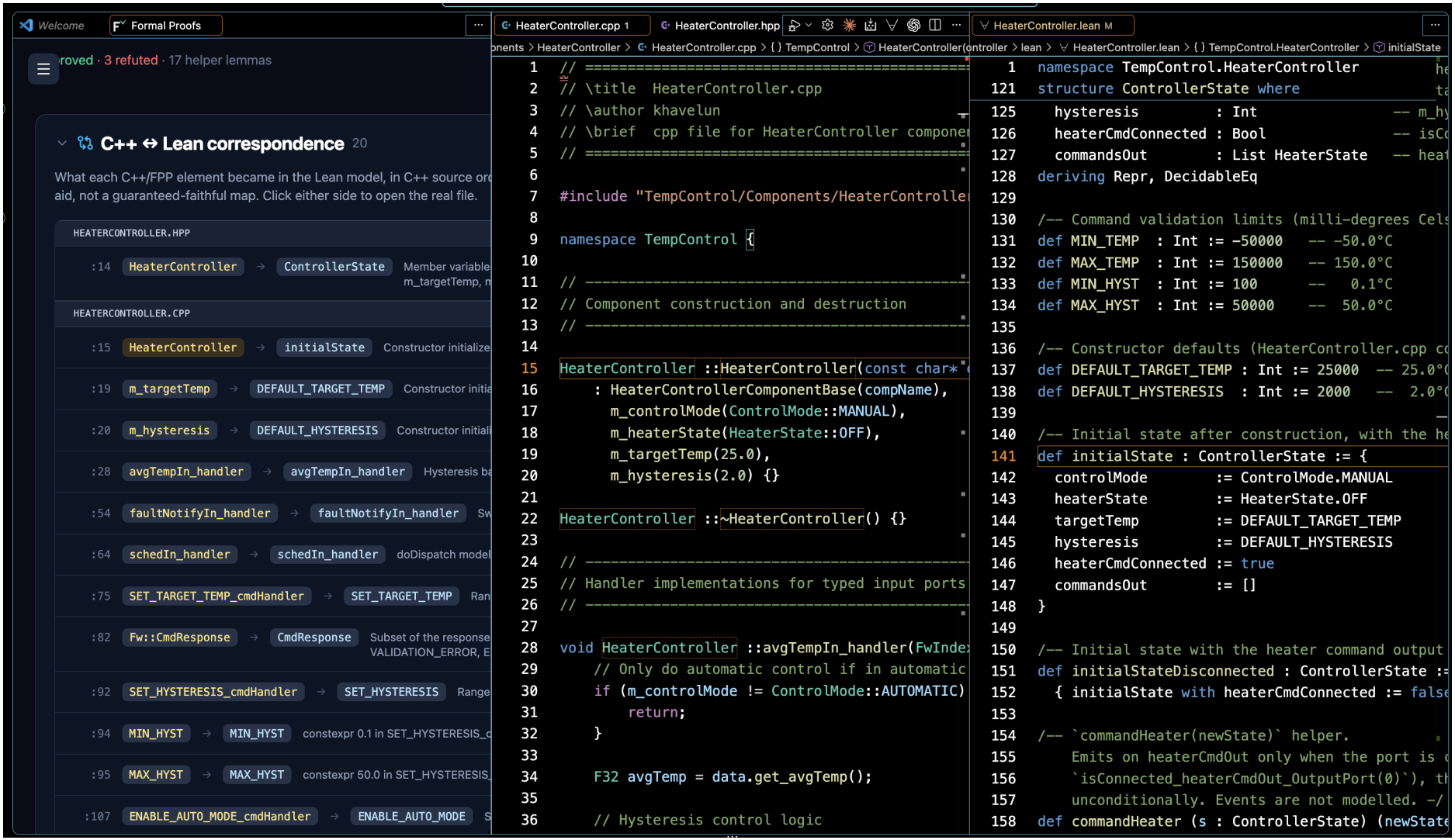
The C++ ↔ LEAN correspondence



HEATERCONTROLLER.HPP			
:14	HeaterController	ControllerState	Member variables m_controlMode, m_heaterState, m_targetTemp, m_hysteresis plus modeled output history.
HEATERCONTROLLER.CPP			
:15	HeaterController	initialState	Constructor initializer-list state; port assumed connected.
:19	m_targetTemp	DEFAULT_TARGET_TEMP	Constructor initializer m_targetTemp(25.0).
:20	m_hysteresis	DEFAULT_HYSTERESIS	Constructor initializer m_hysteresis(2.0).
:28	avgTempIn_handler	avgTempIn_handler	Hysteresis band control in AUTOMATIC mode.
:54	faultNotifyIn_handler	faultNotifyIn_handler	Switches AUTOMATIC to MANUAL on fault.
:64	schedIn_handler	schedIn_handler	doDispatch modeled as identity (assumption).
:75	SET_TARGET_TEMP_cmdHandler	SET_TARGET_TEMP	Range-validated target update with response.
:82	Fw::CmdResponse	CmdResponse	Subset of the responses sent via cmdResponse_out (OK, VALIDATION_ERROR, EXECUTION_ERROR).
:92	SET_HYSTERESIS_cmdHandler	SET_HYSTERESIS	Range-validated hysteresis update with response.
:94	MIN_HYST	MIN_HYST	constexpr 0.1 in SET_HYSTERESIS_cmdHandler, as milli-degrees.
:95	MAX_HYST	MAX_HYST	constexpr 50.0 in SET_HYSTERESIS_cmdHandler, as milli-degrees.
:107	ENABLE_AUTO_MODE_cmdHandler	ENABLE_AUTO_MODE	Sets control mode; always OK.
:117	HEATER_MANUAL_CMD_cmdHandler	HEATER_MANUAL_CMD	Commands heater only in MANUAL mode, else EXECUTION_ERROR.
:136	commandHeater	commandHeater	Guarded heaterCmdOut_out then m_heaterState update; events omitted.
TEMPCONTROLTYPES.FPP			
:4	HeaterState	HeaterState	Enum type of m_heaterState; defined in TempControlTypes.fpp (OFF, ON, FAULT).
:11	ControlMode	ControlMode	Enum type of m_controlMode; defined in TempControlTypes.fpp (MANUAL, AUTOMATIC).
:17	AvgTempData	AvgTempData	Struct parameter of avgTempIn_handler; only avgTemp is read by the C++.
TEMPAVERAGER.FPP			
:89	MIN_TEMP	MIN_TEMP	constexpr -50.0 in SET_TARGET_TEMP_cmdHandler, as milli-degrees.
:92	MAX_TEMP	MAX_TEMP	constexpr 150.0 in SET_TARGET_TEMP_cmdHandler, as milli-degrees.

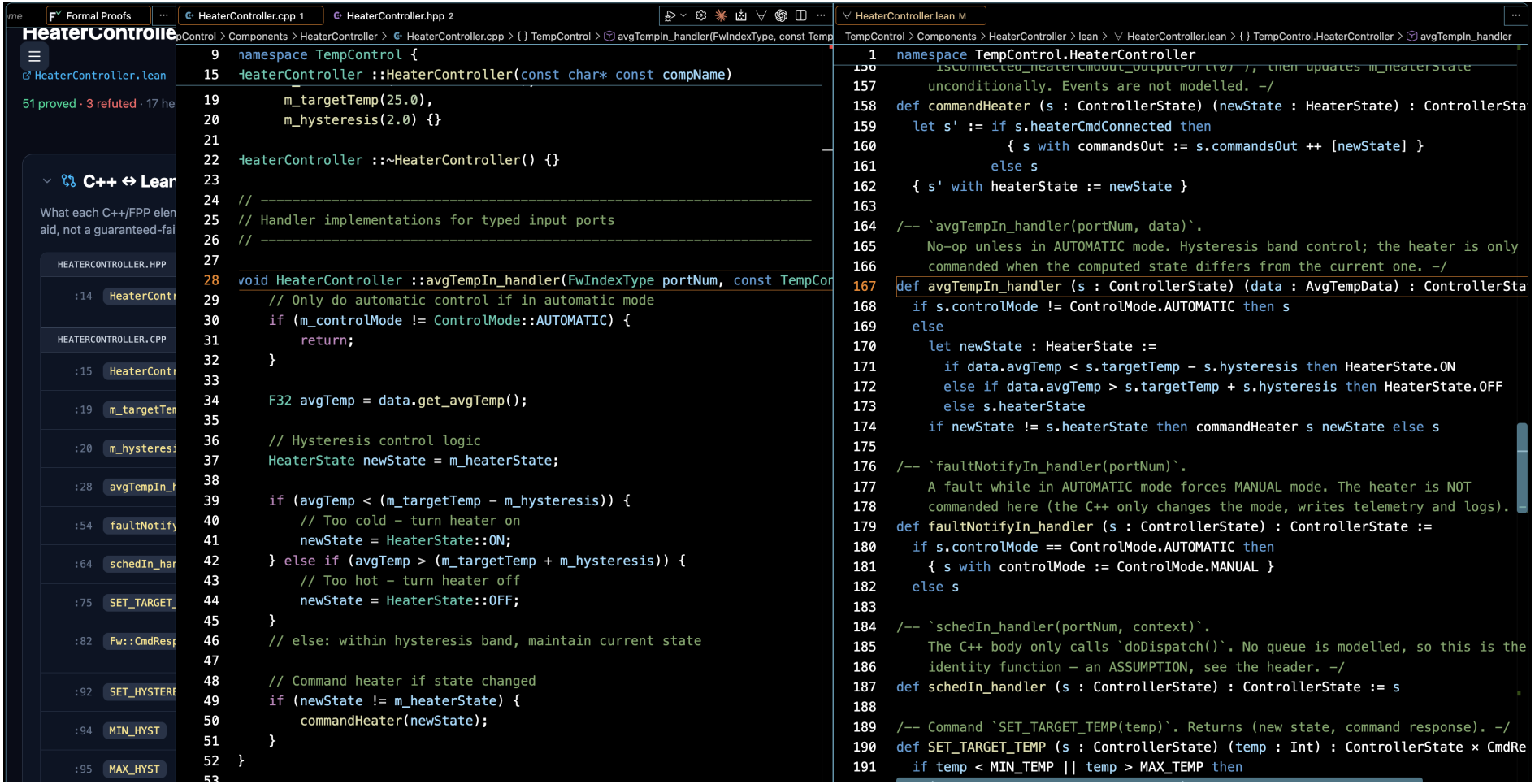
What each C++ element became in the model.

Clicking a row opens both sides



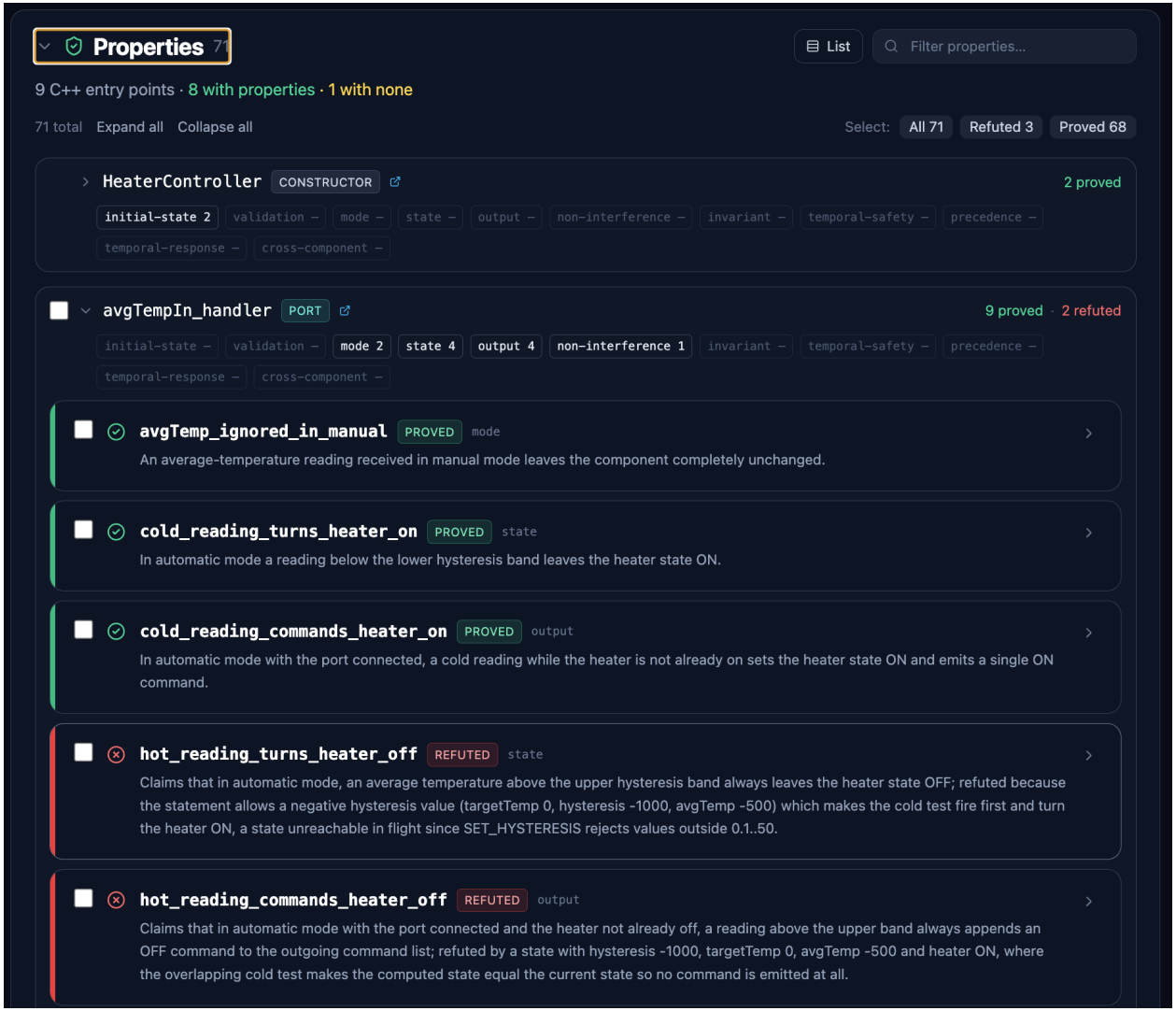
The C++ constructor and the Lean `initialState`, side by side.

The handler and its model



`avgTempIn_handler` in C++ and in Lean: the model tracks the code line for line.

Properties, grouped by C++ entry point



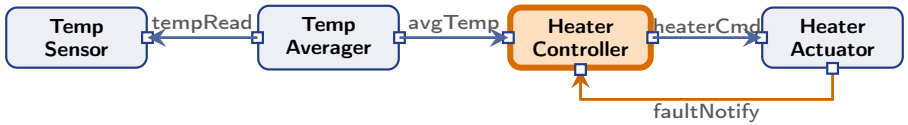
Each with its English, its kind and its status. Two of these are refuted.

A refuted property, expanded



The statement, the counterexample, why the model allows it, and what it means for the C++.

Recall: the heater controller's hysteresis

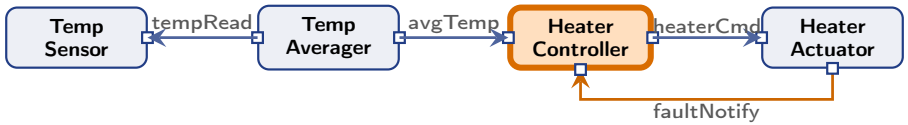


In automatic mode, every average reading goes through two tests:

```
if      (avgTemp < (m_targetTemp - m_hysteresis)) newState = ON;   // too cold
else if (avgTemp > (m_targetTemp + m_hysteresis)) newState = OFF;  // too hot
if (newState != m_heaterState) commandHeater(newState);
```

With the defaults, target 25 °C and hysteresis +2 °C:

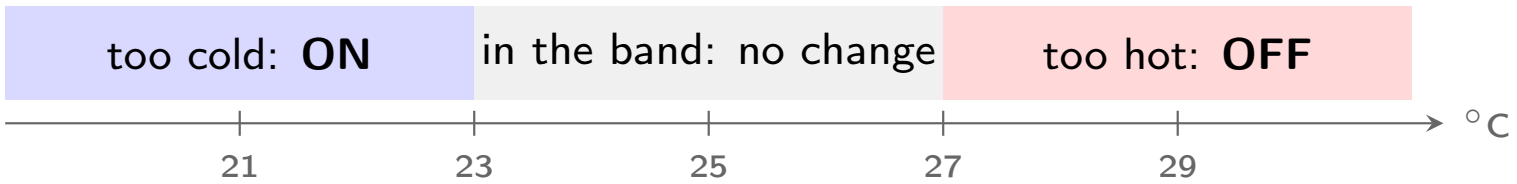
Recall: the heater controller's hysteresis



In automatic mode, every average reading goes through two tests:

```
if      (avgTemp < (m_targetTemp - m_hysteresis)) newState = ON;   // too cold
else if (avgTemp > (m_targetTemp + m_hysteresis)) newState = OFF;  // too hot
if (newState != m_heaterState) commandHeater(newState);
```

With the defaults, target 25 °C and hysteresis +**2** °C:



A proof attempt that failed — usefully

Proposed: *"a hot reading turns the heater off"*.

```
theorem hot_reading_commands_heater_off (s : ControllerState) (d : AvgTempData)
    (ha : s.controlMode = ControlMode.AUTOMATIC)
    (hc : s.heaterCmdConnected = true)
    (hon : s.heaterState ≠ HeaterState.OFF)
    (hhot : s.targetTemp + s.hysteresis < d.avgTemp) :
    let s' := avgTempIn_handler s d
    s'.commandsOut = s.commandsOut ++ [HeaterState.OFF]
```

Refuted, with a counterexample: target 25 °C, hysteresis −2 °C, heater ON, reading 26 °C.

- 26 °C is hot. But $26 < 25 - (-2)$: it also counts as "too cold", that test comes first, and the heater stays **on**.
- The C++ avoids this by checking the hysteresis when it is set: `SET_HYSTERESIS` rejects anything below 0.1 °C. An invariant is needed: `hysteresis > 0`. However, it suggests a **precondition** for the handler.

A refutation is a finding either way: about the code, or about what we believed of it.

A proof attempt that failed — usefully

Proposed: *"a hot reading turns the heater off"*.

```
theorem hot_reading_commands_heater_off (s : ControllerState) (d : AvgTempData)
    (ha : s.controlMode = ControlMode.AUTOMATIC)
    (hc : s.heaterCmdConnected = true)
    (hon : s.heaterState ≠ HeaterState.OFF)
    (hhot : s.targetTemp + s.hysteresis < d.avgTemp) :
    let s' := avgTempIn_handler s d
    s'.commandsOut = s.commandsOut ++ [HeaterState.OFF]
```

Refuted, with a counterexample: target 25 °C, hysteresis −2 °C, heater ON, reading 26 °C.

- 26 °C is hot. But $26 < 25 - (-2)$: it also counts as "too cold", that test comes first, and the heater stays **on**.
- The C++ avoids this by checking the hysteresis when it is set: `SET_HYSTERESIS` rejects anything below 0.1 °C. An invariant is needed: `hysteresis > 0`. However, it suggests a **precondition** for the handler.

A refutation is a finding either way: about the code, or about what we believed of it.

A proof attempt that failed — usefully

Proposed: *"a hot reading turns the heater off"*.

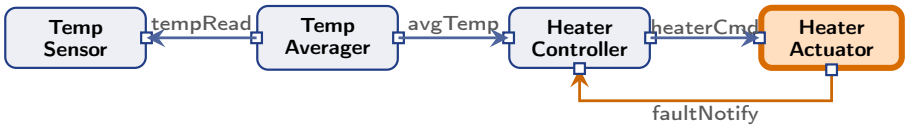
```
theorem hot_reading_commands_heater_off (s : ControllerState) (d : AvgTempData)
    (ha : s.controlMode = ControlMode.AUTOMATIC)
    (hc : s.heaterCmdConnected = true)
    (hon : s.heaterState ≠ HeaterState.OFF)
    (hhot : s.targetTemp + s.hysteresis < d.avgTemp) :
    let s' := avgTempIn_handler s d
    s'.commandsOut = s.commandsOut ++ [HeaterState.OFF]
```

Refuted, with a counterexample: target 25 °C, hysteresis −2 °C, heater ON, reading 26 °C.

- 26 °C is hot. But $26 < 25 - (-2)$: it also counts as "too cold", that test comes first, and the heater stays **on**.
- The C++ avoids this by checking the hysteresis when it is set: `SET_HYSTERESIS` rejects anything below 0.1 °C. An invariant is needed: `hysteresis > 0`. However, it suggests a **precondition** for the handler.

A refutation is a finding either way: about the code, or about what we believed of it.

How the actuator keeps track of the heater



From the controller to the **actuator**: the component that stops the heater when it runs too long. Once a second it records whether the heater is on.

duty cycle

the last 120 seconds, on or off



share of seconds ON: **limit 80%**

18 s

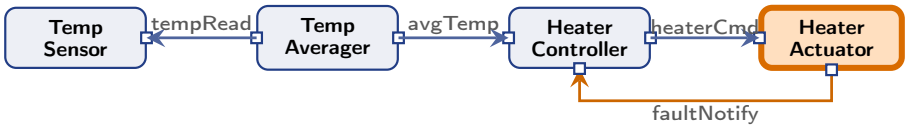
continuous on-time
seconds ON in a row, reset when OFF: **limit 60 s**

Either limit exceeded: the heater goes to **FAULT**.

RESET_STATISTICS, an operator command

Clears the duty-cycle buffer (the last 120 seconds) and sets the on-time counter to 0: the record of how long the heater has been on is gone.

How the actuator keeps track of the heater



From the controller to the **actuator**: the component that stops the heater when it runs too long. Once a second it records whether the heater is on.

duty cycle

the last 120 seconds, on or off



share of seconds ON: **limit 80%**

18 s

continuous on-time
seconds ON in a row, reset when OFF: **limit 60 s**

Either limit exceeded: the heater goes to **FAULT**.

RESET_STATISTICS, an operator command

Clears the duty-cycle buffer (the last 120 seconds) and sets the on-time counter to 0: the record of how long the heater has been on is gone.

A proved theorem that was the wrong theorem

2 LEAN bug found

1. A violated safety limit leads to FAULT within one second (10 ticks):

```
theorem tick_sequence_corrects_safety (s) (params) :
  Reachable s params → violatesSafetyLimits s params →
  let ticks := List.replicate TICKS_PER_UPDATE SystemEvent.Tick
  (executeTrace params ticks s).currentState = HeaterState.FAULT
```

Proved. But the 10 events are *only* ticks: nothing else may happen.

2. Corrected: the heater reaches **FAULT** for *any* events, as long as 10 of them are ticks.


```
theorem safety_response_within_one_second (s) (params) (trace) :
  Reachable s params → violatesSafetyLimits s params →
  countTicks trace = TICKS_PER_UPDATE → ReachesFault s params trace
```

False. `RESET_STATISTICS`, sent while the heater is ON, clears the safety counters but leaves the heater on: repeated, FAULT never comes, and the heater stays on **forever**.

A proved theorem that was the wrong theorem

2

LEAN

bug found

1. A violated safety limit leads to FAULT within one second (10 ticks):

```
theorem tick_sequence_corrects_safety (s) (params) :
  Reachable s params → violatesSafetyLimits s params →
  let ticks := List.replicate TICKS_PER_UPDATE SystemEvent.Tick
  (executeTrace params ticks s).currentState = HeaterState.FAULT
```

Proved. But the 10 events are *only* ticks: nothing else may happen.

2. Corrected: the heater reaches **FAULT** for *any* events, as long as 10 of them are ticks.



```
theorem safety_response_within_one_second (s) (params) (trace) :
  Reachable s params → violatesSafetyLimits s params →
  countTicks trace = TICKS_PER_UPDATE → ReachesFault s params trace
```

False. `RESET_STATISTICS`, sent while the heater is ON, clears the safety counters but leaves the heater on: repeated, FAULT never comes, and the heater stays on **forever**.

A proved theorem that was the wrong theorem

2 LEAN bug found

1. A violated safety limit leads to FAULT within one second (10 ticks):

```
theorem tick_sequence_corrects_safety (s) (params) :
  Reachable s params → violatesSafetyLimits s params →
  let ticks := List.replicate TICKS_PER_UPDATE SystemEvent.Tick
  (executeTrace params ticks s).currentState = HeaterState.FAULT
```

Proved. But the 10 events are *only* ticks: nothing else may happen.

2. Corrected: the heater reaches **FAULT** for *any* events, as long as 10 of them are ticks.



```
theorem safety_response_within_one_second (s) (params) (trace) :
  Reachable s params → violatesSafetyLimits s params →
  countTicks trace = TICKS_PER_UPDATE → ReachesFault s params trace
```

False. `RESET_STATISTICS`, sent while the heater is ON, clears the safety counters but leaves the heater on: repeated, FAULT never comes, and the heater stays on **forever**.

The fix, and the third theorem

3. Fixed in the C++: the reset is rejected while ON.

```
void HeaterActuator::RESET_STATISTICS_cmdHandler(...) {
    if (m_currentState == HeaterState::ON) { ... EXECUTION_ERROR; return; }
    ...                                    // only now clear the counters
```

And the goal widened, from ReachesFault to SafetyResolved: FAULT, *or* the violation gone.

```
theorem safety_response_within_one_second (s) (params) (trace) :
  Reachable s params → violatesSafetyLimits s params →
  countTicks trace = TICKS_PER_UPDATE → SafetyResolved s params trace
```

Proved.

The fix, and the third theorem

3. Fixed in the C++: the reset is rejected while ON.

```
void HeaterActuator::RESET_STATISTICS_cmdHandler(...) {
    if (m_currentState == HeaterState::ON) { ... EXECUTION_ERROR; return; }
    ...                                    // only now clear the counters
```

And the goal widened, from `ReachesFault` to `SafetyResolved`: FAULT, *or* the violation gone.

```
theorem safety_response_within_one_second (s) (params) (trace) :
  Reachable s params → violatesSafetyLimits s params →
  countTicks trace = TICKS_PER_UPDATE → SafetyResolved s params trace
```

Proved.

What does "resolved" mean?

The heater faults, *or* the violation is gone:

```
def SafetyResolved (s) (params) (trace) : Prop :=
  exists pre suf, trace = pre ++ suf /\
    ((executeTrace params pre s).currentState = HeaterState.FAULT \/
     not (violatesSafetyLimits (executeTrace params pre s) params))
```

"Gone" is measured on the **counters**: the duty-cycle buffer and the on-time.

- `RESET_STATISTICS` sets both to 0, so the violation is "gone", while the heater is still on.
- So the definition cannot tell a heater turned OFF from **wiped evidence**.

The unfixed code satisfies this theorem too

The property was corrected, then weakened again: it no longer excludes the bug. Only the C++ fix does, by rejecting the reset while ON.

What does "resolved" mean?

The heater faults, *or* the violation is gone:

```
def SafetyResolved (s) (params) (trace) : Prop :=
  exists pre suf, trace = pre ++ suf /\
    ((executeTrace params pre s).currentState = HeaterState.FAULT \/
     not (violatesSafetyLimits (executeTrace params pre s) params))
```

"Gone" is measured on the **counters**: the duty-cycle buffer and the on-time.

- `RESET_STATISTICS` sets both to 0, so the violation is "gone", while the heater is still on.
- So the definition cannot tell a heater turned OFF from **wiped evidence**.

The unfixed code satisfies this theorem too

The property was corrected, then weakened again: it no longer excludes the bug. Only the C++ fix does, by rejecting the reset while ON.

What does "resolved" mean?

The heater faults, *or* the violation is gone:

```
def SafetyResolved (s) (params) (trace) : Prop :=
  exists pre suf, trace = pre ++ suf /\
    ((executeTrace params pre s).currentState = HeaterState.FAULT \/
     not (violatesSafetyLimits (executeTrace params pre s) params))
```

"Gone" is measured on the **counters**: the duty-cycle buffer and the on-time.

- `RESET_STATISTICS` sets both to 0, so the violation is "gone", while the heater is still on.
- So the definition cannot tell a heater turned OFF from **wiped evidence**.

The unfixed code satisfies this theorem too

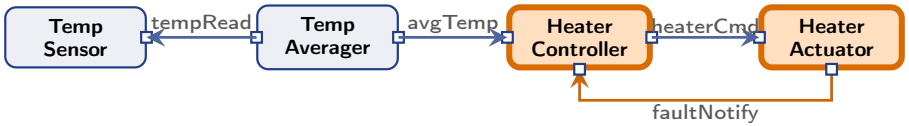
The property was corrected, then weakened again: it no longer excludes the bug. Only the C++ fix does, by rejecting the reset while ON.

The whole system, composed along the topology

3

LEAN

proved





→ A system model from `topology.fpp`. The controller is modelled in full; the actuator is **abstracted** to OFF, ON or FAULT, and may fault at any time.

```
structure SystemState where
  sensor : Sensor.State
  averager : Averager.State
  controller : Controller.State
  actuator : Actuator.State
```

```
theorem system_step_fault_forces_manual_mode (sys) (p) (hi lo) :
  (systemStep sys p true hi lo).controller.controlMode = manual
```

An actuator fault puts the controller in manual mode, from *any* state.

The Actuator abstraction was chosen by the LLM for convenience (no Mathlib): sound for the controller's reaction to faults, but blind to what the actuator does after one as we shall see next.

Instrumenting the generated code for monitoring

F′ generates a C++ class per port type. In a verification build, each one gets a subclass that records the call, then forwards it:

```
class OutputHeaterCommandPort : public OutputHeaterCommandPortBase {
  public:
    void invoke(const TempControl::HeaterState& heaterState) const {
      Fcheck::record(getObjName(), heaterState);
      OutputHeaterCommandPortBase::invoke(heaterState);
    }
};
```

The raw record: time, port, event id, timestamp, severity, argument bytes:

```
92281991262,heaterActuator_eventOut_OutputPort[0],536903680,
1792172611.792976,x06,x0000000200000001
```

translated with the F′ dictionary into a named event:

```
heaterActuator_STATE_CHANGED("FAULT", "ON")
```

Instrumenting the generated code for monitoring

F′ generates a C++ class per port type. In a verification build, each one gets a subclass that records the call, then forwards it:

```
class OutputHeaterCommandPort : public OutputHeaterCommandPortBase {
  public:
    void invoke(const TempControl::HeaterState& heaterState) const {
      Fcheck::record(getObjName(), heaterState);
      OutputHeaterCommandPortBase::invoke(heaterState);
    }
};
```

The raw record: time, port, event id, timestamp, severity, argument bytes:

```
92281991262,heaterActuator_eventOut_OutputPort[0],536903680,
1792172611.792976,x06,x0000000200000001
```

translated with the F′ dictionary into a named event:

```
heaterActuator_STATE_CHANGED("FAULT", "ON")
```

Instrumenting the generated code for monitoring

F′ generates a C++ class per port type. In a verification build, each one gets a subclass that records the call, then forwards it:

```
class OutputHeaterCommandPort : public OutputHeaterCommandPortBase {
  public:
    void invoke(const TempControl::HeaterState& heaterState) const {
      Fcheck::record(getObjName(), heaterState);
      OutputHeaterCommandPortBase::invoke(heaterState);
    }
};
```

The raw record: time, port, event id, timestamp, severity, argument bytes:

```
92281991262,heaterActuator_eventOut_OutputPort[0],536903680,
1792172611.792976,x06,x0000000200000001
```

translated with the F′ dictionary into a named event:

```
heaterActuator_STATE_CHANGED("FAULT", "ON")
```

A temporal property, in DejaVuMT

4

DejaVuMT

violated

Using the event `STATE_CHANGED` from the generated vocabulary (slide 7):

```
// The actuator never goes straight from FAULT to ON: FAULT goes through OFF.
prop fault_cleared_before_heating :
  !heaterActuator_STATE_CHANGED("FAULT", "ON")
```

The violation, on a recorded run

Scenario: heat until the heater has been on too long, then turn it on by hand.

```
70.870 s  heaterActuator_STATE_CHANGED(ON, FAULT)
70.870 s  faultNotify
71.281 s  heaterController_MODE_CHANGED(MANUAL)
77.846 s  cmd_heaterController_HEATER_MANUAL_CMD(ON)
77.846 s  heaterCmd(ON)
77.846 s  heaterActuator_STATE_CHANGED(FAULT, ON)    <- violated here
```

A real hazard

The controller, in manual mode, allows the operator's ON straight on from FAULT. The heater now runs with its **safety limits off**.

The same kind of bug as RESET_STATISTICS

An operator command gets around the fault protection. The fix for the first bug, found with Lean, did not cover this path.

The violation, on a recorded run

Scenario: heat until the heater has been on too long, then turn it on by hand.

```
70.870 s  heaterActuator_STATE_CHANGED(ON, FAULT)
70.870 s  faultNotify
71.281 s  heaterController_MODE_CHANGED(MANUAL)
77.846 s  cmd_heaterController_HEATER_MANUAL_CMD(ON)
77.846 s  heaterCmd(ON)
77.846 s  heaterActuator_STATE_CHANGED(FAULT, ON)    <- violated here
```

A real hazard

The controller, in manual mode, allows the operator's ON straight on from FAULT. The heater now runs with its **safety limits off**.

The same kind of bug as RESET_STATISTICS

An operator command gets around the fault protection. The fix for the first bug, found with Lean, did not cover this path.

The violation, on a recorded run

Scenario: heat until the heater has been on too long, then turn it on by hand.

```
70.870 s  heaterActuator_STATE_CHANGED(ON, FAULT)
70.870 s  faultNotify
71.281 s  heaterController_MODE_CHANGED(MANUAL)
77.846 s  cmd_heaterController_HEATER_MANUAL_CMD(ON)
77.846 s  heaterCmd(ON)
77.846 s  heaterActuator_STATE_CHANGED(FAULT, ON)    <- violated here
```

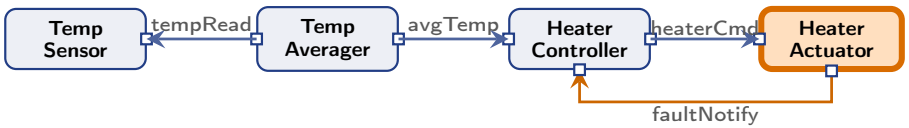
A real hazard

The controller, in manual mode, allows the operator's ON straight on from FAULT. The heater now runs with its **safety limits off**.

The same kind of bug as `RESET_STATISTICS`

An operator command gets around the fault protection. The fix for the first bug, found with Lean, did not cover this path.

Why FAULT → ON is dangerous: the latches



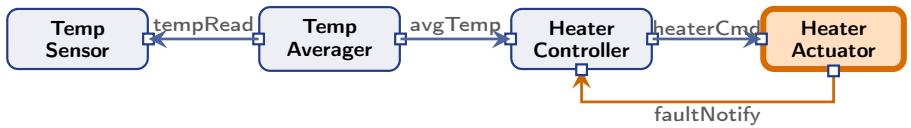
Each limit has a **latch**: a flag that, once set, stays set. A set latch means "this fault has already been reported", so the check does not fire every second.

```
if (dutyCycle > maxDutyCycle && !m_dutyCycleFaultTriggered) {
    setFaultState();                          // the heater goes to FAULT
    m_dutyCycleFaultTriggered = true;         // reported: stay silent from now on
}
...
if (oldState == FAULT && newState == OFF) {
    m_dutyCycleFaultTriggered = false;        // only FAULT -> OFF re-arms it
    m_continuousOnTimeFaultTriggered = false;
}
```

The expected flow: FAULT → OFF → ON

Going through OFF clears the latches, so both limits are checked again once the heater is back on.

Why FAULT → ON is dangerous: the latches



Each limit has a **latch**: a flag that, once set, stays set. A set latch means "this fault has already been reported", so the check does not fire every second.

```
if (dutyCycle > maxDutyCycle && !m_dutyCycleFaultTriggered) {
    setFaultState();                          // the heater goes to FAULT
    m_dutyCycleFaultTriggered = true;         // reported: stay silent from now on
}
...
if (oldState == FAULT && newState == OFF) {
    m_dutyCycleFaultTriggered = false;        // only FAULT -> OFF re-arms it
    m_continuousOnTimeFaultTriggered = false;
}
```

The expected flow: FAULT → OFF → ON

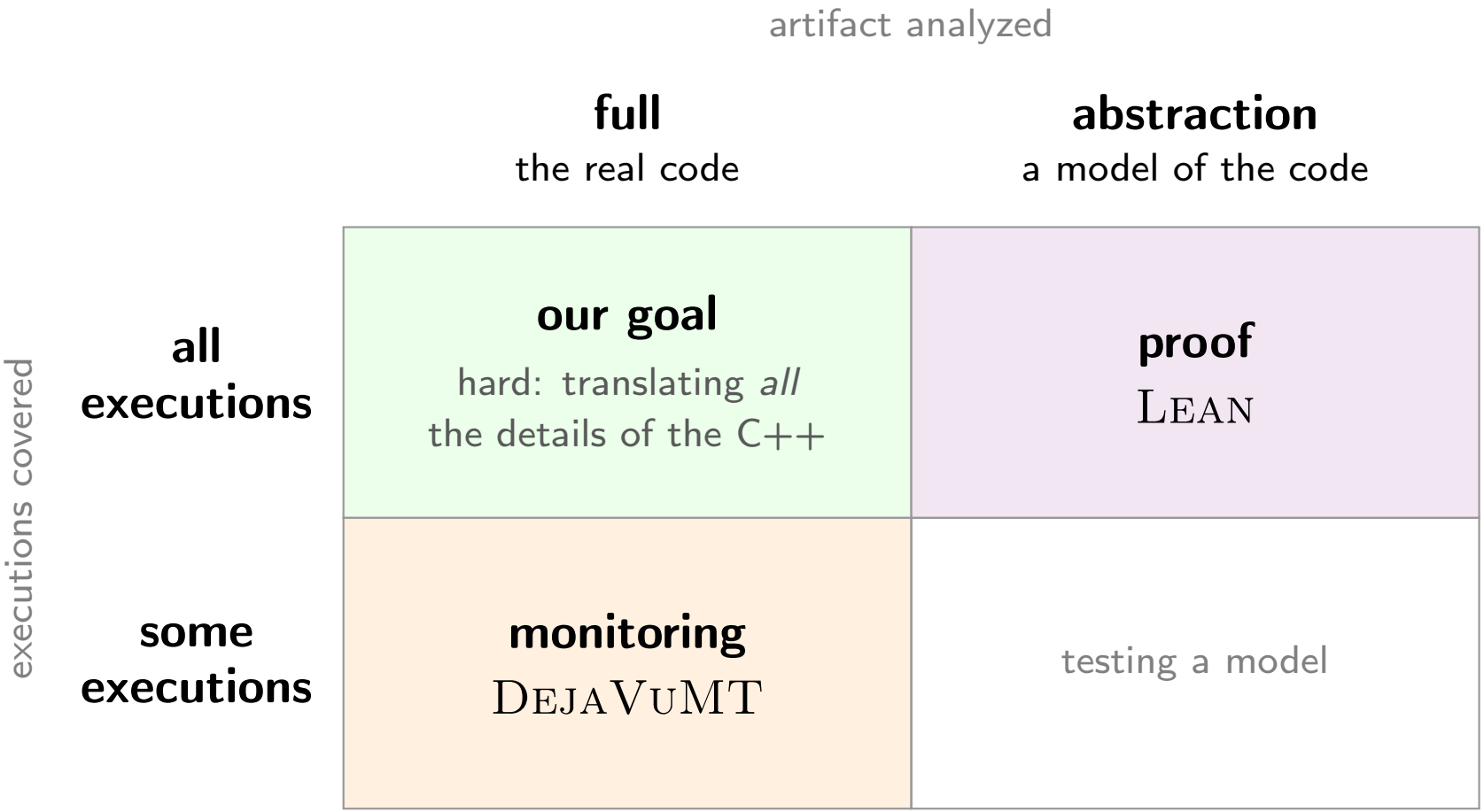
Going through OFF clears the latches, so both limits are checked again once the heater is back on.

Four properties, three levels

	checked on	property	holds?
1	LEAN controller model	a hot reading turns the heater off	**false** as stated; **true** given the proved invariant `hysteresis > 0`
2	LEAN actuator model	a violated limit leads to FAULT, whatever else happens	**false**: operator sends `RESET_STATISTICS`
3	LEAN system model	a fault puts the controller in manual mode	**true**
4	DEJAVUMT the real code	the heater never goes directly from FAULT to ON	**false**: operator sends `HEATER_MANUAL_CMD(ON)`

2 and 4: an operator command gets around the fault protection.

What each method covers



Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Conclusion

1. LEAN proofs went through automatically, which is a game changer - so far.
2. But writing the specification is still hard:
   - which properties to write; One has to know what the "knobs" are and how to turn them.
   - getting a single property right. The `RESET_STATISTICS` bug hid behind a proved but too weak property.
3. Properties are associated with natural language explanations.
4. Is the model the code? The FAULT $\rightarrow$ ON bug hid behind a proved but too weak model. Tricky problems include memory safety, aliasing, and concurrency.
5. One can also use the LLM to find problems in the code and generate a model, e.g. in Promela (Spin) to demonstrate it.
6. An LLM can perform static analysis, checking against coding standards.
7. One can of write the spec first, then the code, and then prove the code against the spec.

Questions?